

Week 2 Module

1. Fraud Transaction Detection (Linear Search)

Problem Statement

→ A bank stores transaction IDs in the order they occur. Since the transactions are not stored, the security team needs to check if a suspicious transactions ID exists.

Write a program using linear search.

→ Input Format

- First line contains integer n
- Second line contains n transaction IDs
- Third line contains suspicious transaction ID.

→ Output Format

Print:

Fraud Transaction Found

or

Transaction Not Found

→ Example

Input

5

1201 1305 1450 1408 1502

1450

Output

Fraud Transaction Found.

→ Logic:

- Read n transaction ID into an array.
- Use linear search to check each transaction.
- If suspicious found, print "Fraud Transaction Found" else print "Transaction Not Found."

#include <iostream>
using namespace std;

int main()

{
int N;

cin >> N;

int transactionID[N];

for (int i = 0; i < N; i++)
cin >> transactionID[i];

}

int suspiciousID;

cin >> suspiciousID;

bool found = false;

for (int i = 0; i < N; i++)

{
if (transactionID[i] == suspiciousID)

{

found = true;

break;

}

}

if (found)

{
cout << "Fraud Transaction Found";

else

cout << "Transaction Not Found";

return 0;

}

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4      int N;
5      cin >> N;
6
7      int transactionID[N];
8      for(int i=0;i<N;i++)
9      {
10         cin >> transactionID[i];
11     }
12
13     int suspiciousID;
14     cin >> suspiciousID;
15     bool found=false;
16
17     for(int i=0;i<N;i++)
18     {
19         if (transactionID[i] == suspiciousID)
20         {
21             found=true;
22             break;
23         }
24     }
25     if (found)
26         cout << "Fraud Transaction Found";
27     else
28         cout << "Transaction Not Found";
29
30     return 0;
31 }
```

```
5
1201 1305 1450 1408 1502
1450
Fraud Transaction Found
```

2. Server log Search (Binary Search)

Problem Statement

→ A cloud company stores server log IDs in sorted order. Engineers need to quickly verify if a log ID exists.

Write a program using Binary Search.

→ Input Format

- First line contains N
- Second line contains sorted log IDs.
- Third line contains target log ID.

→ Output Format

Print:

log Found

or

log Not Found

Constraints

- $1 \leq N \leq 10^5$

→ Example

Input

7

1001 1005 1010 1015 1020 1030 1040

1015

Output

log Found.

→ Logic:

- Read N & the sorted log IDs into an array.
- Use binary search,
 - i) First, find middle element, if it matches to target, then print "log Found".
 - ii) If target is greater, search the right half of the array.
 - iii) otherwise, search the left half of the array.
- If element is not found, print "log not found".

```
#include <iostream>
using namespace std;
```

```
int main() {
    int N;
    cin >> N;
```

```
    int log[N];
    for (int i=0; i<N; i++) {
        cin >> log[i];
    }
```

```
    int target;
    cin >> target;
```

```
    int low = 0, high = N-1;
    bool found = false;
```

```
    while (low <= high) {
        int mid = low + (high - low) / 2;
```

```
if (log[mid] == target)
```

```
{  
    found = true;
```

```
    break;
```

```
}
```

```
else if (log[mid] < target)
```

```
{
```

```
    low = mid + 1;
```

```
}
```

```
else
```

```
{
```

```
    high = mid - 1;
```

```
}
```

```
}
```

```
if (found)
```

```
    cout << "log Found";
```

```
else
```

```
    cout << "log Not Found";
```

```
return 0;
```

```
}
```

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4      int N;
5      cin >> N;
6      int log[N];
7      for(int i=0;i<N;i++) {
8          cin >> log[i];
9      }
10     int target;
11     cin >> target;
12     int low=0, high=N-1;
13     bool found = false;
14     while(low <= high) {
15         int mid=low+(high-low)/2;
16         if (log[mid] == target) {
17             found = true;
18             break;
19         }
20         else if (log[mid] < target) {
21             low = mid + 1;
22         }
23         else {
24             high = mid - 1;
25         }
26     }
27     if (found)
28         cout << "Log Found";
29     else
30         cout << "Log Not Found";
31
32     return 0;
33 }
```

```
7
1001 1005 1010 1015 1020 1030 1040
1015
Log Found
```

3. Sort Employee Salaries (Bubble Sort)

• Problem Statement

→ A company wants to sort employee salaries in ascending order for payroll analysis.

Write a program using bubble sort.

→ Input Format

- First line contains N
- Second line contains salaries

→ Output Format

Print sorted salaries

→ Example

Input

5

45000 32000 55000 28000 60000

Output

28000 32000 45000 55000 60000

→ Logic:

- Read N & salaries into an array.
- Use bubble sort,
 - i) Compare adjacent elements
 - ii) Swap them, if they are in not wrong order
 - iii) Repeat until the array is sorted.
- Print sorted salaries.

```
#include <iostream>
using namespace std;
```

```
int main()
```

```
{
    int N;
```

```
    cin >> N;
```

```
    int salaries[N];
```

```
    for (int i=0; i<N; i++)
```

```
        cin >> salaries[i];
```

```
}
```

```
    for (int i=0; i<N-1; i++)
```

```
        for (int j=0; j<N-i-1; j++)
```

```
{
```

```
            if (salaries[j] > salaries[j+1])
```

```
{
```

```
                int temp = salaries[j];
```

```
                salaries[j] = salaries[j+1];
```

```
                salaries[j+1] = temp;
```

```
            }
```

```
        }
```

```
    }
```

```
    for (int i=0; i<N; i++)
```

```
{
```

```
        cout << salaries[i] << " ";
```

```
}
```

```
    return 0;
```

```
}
```

```
1 #include <iostream>
2 using namespace std;
3 int main() {
4     int N;
5     cin >> N;
6     int salaries[N];
7
8     for(int i=0;i<N;i++) {
9         cin >> salaries[i];
10    }
11
12    for(int i=0;i<N-1;i++)
13    {
14        for(int j=0;j<N-i-1;j++)
15        {
16            if (salaries[j] > salaries[j+1])
17            {
18                int temp=salaries[j];
19                salaries[j]=salaries[j+1];
20                salaries[j+1]=temp;
21            }
22        }
23    }
24    for(int i=0;i<N; i++)
25    {
26        cout << salaries[i] << " ";
27    }
28    return 0;
29 }
```

```
5
45000 32000 55000 28000 60000
28000 32000 45000 55000 60000
```

4. Sort Website Response Time (Insertion Sort)

Problem Statement

→ A web monitoring system records website response time. New data comes continuously and must be inserted into the correct position.

Write a program using Insertion Sort.

→ Input Format

- First line contains N
- Second line contains response times.

→ Output Format

Print sorted response times.

→ Example

Input

5

250 120 320 180 100

Output

100 120 180 250 320

→ Logic:

- Start from the second element ($i=1$).
- Store current element as Key.
- Compare Key with the elements before it.
- Shift larger elements one position to the right.
- Insert Key at its correct position.
- Repeat until the entire array is sorted.

```
#include <iostream>
using namespace std;
```

```
int main () {
```

```
    int n;
```

```
    cin >> n;
```

```
    int responseTime[n];
```

```
    for (int i=0; i<n; i++) {
```

```
        cin >> responseTime[i];
```

```
    }
```

```
    for (int i=1; i<n; i++)
```

```
    {
```

```
        int key = responseTime[i];
```

```
        int j = i-1;
```

```
        while (j >= 0 && responseTime[j] > key)
```

```
        {
```

```
            responseTime[j+1] = responseTime[j];
```

```
            j--;
```

```
        }
```

```
        responseTime[j+1] = key;
```

```
    }
```

```
    for (int i=0; i<n; i++)
```

```
    {
```

```
        cout << responseTime[i] << " ";
```

```
    }
```

```
    return 0;
```

```
}
```

```
1 #include <iostream>
2 using namespace std;
3 int main() {
4     int N;
5     cin >> N;
6     int responseTime[N];
7     for(int i=0;i<N;i++)
8     {
9         cin >> responseTime[i];
10    }
11
12    for(int i=1;i<N;i++)
13    {
14        int key=responseTime[i];
15        int j=i-1;
16
17        while (j>=0 && responseTime[j] > key)
18        {
19            responseTime[j+1]=responseTime[j];
20            j--;
21        }
22
23        responseTime[j+1]=key;
24    }
25
26    for(int i=0;i<N;i++)
27    {
28        cout << responseTime[i] << " ";
29    }
30    return 0;
31 }
```

```
5
250 120 320 180 100
100 120 180 250 320
```

5. Prioritize Bug Reports (Selection Sort)

Problem Statement

→ A software company assigns priority scores to bug reports. To resolve bugs efficiently, they want to sort priorities in ascending order.

Write a program using selection sort.

→ Input Format

- First line contains n
- Second line contains priority scores.

→ Output Format

Print sorted priority scores.

→ Example

Input

5

9 2 8 1 5

Output

1 2 5 8 9

→ Logic:

- Assume the first unsorted element is the minimum.
- Search the remaining array to find actual minimum element.
- Swap the minimum element with the first unsorted element.
- Repeat the process for the remaining unsorted part of the array.

```
#include <iostream>
using namespace std;
```

```
int main() {
    int n;
    cin >> n;
```

```
    int priority[n];
    for (int i = 0; i < n; i++) {
        cin >> priority[i];
    }
```

```
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (priority[j] < priority[minIndex]) {
                minIndex = j;
            }
        }
```

```
        int temp = priority[i];
        priority[i] = priority[minIndex];
        priority[minIndex] = temp;
    }
```

```
    for (int i = 0; i < n; i++) {
        cout << priority[i] << " ";
    }
    return 0;
```

```
}
```

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4      int N;
5      cin >> N;
6
7      int priority[N];
8      for(int i=0;i<N;i++) {
9          cin >> priority[i];
10     }
11
12     for(int i=0;i<N-1;i++)
13     {
14         int minIndex=i;
15         for(int j=i+1;j<N;j++)
16         {
17             if (priority[j] < priority[minIndex])
18             {
19                 minIndex = j;
20             }
21         }
22
23         int temp=priority[i];
24         priority[i]=priority[minIndex];
25         priority[minIndex]=temp;
26     }
27
28     for(int i=0;i<N;i++)
29     {
30         cout << priority[i] << " ";
31     }
32     return 0;
33 }
```

```
5
9 2 8 1 5
1 2 5 8 9
```